

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

-----□□□□-----



BÁO CÁO CUỐI KÌ

HỌC PHẦN IoT VÀ ỨNG DỤNG

Lớp chuyên ngành: D22CNPM02

ĐỀ TÀI: HỆ THỐNG TƯỚI CÂY THÔNG MINH

Giảng viên hướng dẫn: TS.Kim Ngọc Bách

Danh sách thành viên nhóm:

Phan Văn Thủy B22DCCN844

Lê Vũ Thành Vinh B22DCCN904

Bùi Thu Hằng B22DCCN279

Hoàng Việt Anh B22DCCN017

Hà Nội, 2025

MỤC LỤC

LỜI CẢM ƠN.....	3
CHƯƠNG 1. GIỚI THIỆU CHUNG.....	5
1.1. Lý do chọn đề tài.....	5
1.2. Tổng quan về dự án.....	5
1.3. Mục đích của dự án.....	6
1.4. Đối tượng và phạm vi nghiên cứu.....	6
1.5. Phương pháp nghiên cứu.....	6
CHƯƠNG 2. Nền tảng lý thuyết.....	8
2.1. Module ESP32.....	8
2.2. Các cảm biến sử dụng.....	10
2.3. Thiết bị chấp hành.....	15
2.4. Các giao thức truyền thông.....	18
2.5. Cơ chế cập nhật firmware từ xa (OTA).....	21
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU CHỨC NĂNG.....	23
3.1. Chức năng giám sát môi trường.....	23
3.2. Chức năng điều khiển tưới (Thủ công/Tự động).....	23
3.3. Chức năng cấu hình ngưỡng.....	24
3.4. Chức năng cập nhật Firmware (Check Version & Update).....	24
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG.....	25
4.1. Sơ đồ khối hệ thống.....	25
4.2 Thiết kế phần cứng (Sơ đồ nguyên lý).....	26
4.4. Thiết kế Website quản lý.....	37
4.5. Tích hợp hệ thống và GitHub Repo.....	42
CHƯƠNG 5. KẾT LUẬN.....	45
5.1. Kết quả đạt được.....	45
5.2. Hạn chế.....	45
5.3. Hướng phát triển.....	46

LỜI CẢM ƠN

Trước tiên, nhóm chúng em xin gửi lời cảm ơn đặc biệt và sâu sắc nhất đến thầy Kim Ngọc Bách, giảng viên hướng dẫn môn học, người đã tận tình giảng dạy, truyền đạt kiến thức và đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài “Thiết kế hệ thống tưới cây thông minh”.

Nhờ những bài giảng sinh động, những ví dụ thực tế và sự hướng dẫn chi tiết của thầy, nhóm đã nắm vững kiến thức nền tảng về IoT, lập trình nhúng ESP32, giao thức truyền thông và các kỹ thuật triển khai OTA, từ đó tự tin xây dựng một hệ thống hoàn chỉnh, có tính ứng dụng cao. Sự nhiệt tình, tâm huyết và những góp ý quý báu của thầy chính là nguồn động lực lớn nhất giúp nhóm vượt qua khó khăn và hoàn thiện sản phẩm đúng tiến độ.

Môn học không chỉ giúp chúng em củng cố lý thuyết mà còn rèn luyện kỹ năng thực hành, làm việc nhóm và tư duy sáng tạo – những kỹ năng vô cùng quý giá cho tương lai.

Dù đã cố gắng hết mình, báo cáo và sản phẩm của nhóm chắc chắn vẫn còn những thiếu sót. Chúng em rất mong nhận được sự góp ý và nhận xét từ thầy để tiếp tục hoàn thiện hơn trong giai đoạn cuối kỳ.

Một lần nữa, nhóm xin chân thành cảm ơn thầy Kim Ngọc Bách!

CHƯƠNG 1. GIỚI THIỆU CHUNG

1.1. Lý do chọn đề tài

Trong đời sống hiện đại, nhu cầu trồng cây cảnh, rau sạch tại nhà, trên ban công hoặc trong các khu vườn nhỏ ngày càng tăng cao. Tuy nhiên, việc chăm sóc cây trồng vẫn chủ yếu dựa vào phương pháp tưới nước thủ công theo thói quen hoặc kinh nghiệm cá nhân. Phương pháp này tồn tại nhiều bất cập:

- Tốn nhiều thời gian và công sức, đặc biệt với những người bận rộn hoặc thường xuyên vắng nhà.
- Thiếu tính chính xác, dễ gây thừa nước (úng rề, thối rề) hoặc thiếu nước (cây héo, chậm phát triển).
- Không thể giám sát và điều khiển từ xa khi người chăm sóc đi công tác, du lịch dài ngày.

Với sự phát triển mạnh mẽ của công nghệ IoT, vi điều khiển giá rẻ ESP32 và khả năng lập trình web hiện đại, việc xây dựng một hệ thống tưới cây thông minh hoàn toàn tự chủ, có giao diện web riêng, chi phí thấp và hỗ trợ cập nhật firmware từ xa (OTA) là hoàn toàn khả thi và rất thực tế.

Xuất phát từ nhu cầu thực tiễn trên, nhóm quyết định thực hiện đề tài **“Thiết kế hệ thống tưới cây thông minh tích hợp OTA qua GitHub”** nhằm giải quyết vấn đề chăm sóc cây trồng một cách hiệu quả.

1.2. Tổng quan về dự án

Dự án xây dựng một hệ thống tưới cây tự động thông minh với các thành phần chính:

- Vi điều khiển ESP32 làm trung tâm xử lý và kết nối Wi-Fi.
- Cảm biến độ ẩm đất điện dung, cảm biến nhiệt độ-độ ẩm không khí DHT22, cảm biến ánh sáng (tùy chọn).
- Relay điều khiển bơm nước mini.
- Web Server được nhúng trực tiếp trên ESP32 (dùng ESPAsyncWebServer) và một trang web quản lý độc lập (host trên VPS hoặc GitHub Pages) để giám sát và điều khiển từ xa qua trình duyệt bất kỳ (điện thoại, máy tính).
- Cơ chế OTA (Over-The-Air) cập nhật firmware trực tiếp từ GitHub Releases mà không cần cáp nối.

Hệ thống hỗ trợ 2 chế độ hoạt động:

- Tự động: ESP32 tự đọc cảm biến, so sánh với ngưỡng đã cài đặt và bật/tắt bơm.
- Thủ công: Người dùng điều khiển trực tiếp qua giao diện web.

1.3. Mục đích của dự án

Dự án hướng tới các mục tiêu chính sau:

1. Xây dựng hệ thống tưới cây tự động hoàn toàn độc lập, không phụ thuộc vào bất kỳ nền tảng IoT thương mại nào.
2. Cung cấp giao diện web thân thiện, hoạt động tốt trên cả điện thoại và máy tính, cho phép:
 - Theo dõi realtime độ ẩm đất, nhiệt độ, độ ẩm không khí.
 - Điều khiển thủ công bơm nước.

- Cài đặt ngưỡng độ ẩm và thời gian tưới.
 - Nhận thông báo khi có sự cố.
3. Tích hợp cơ chế OTA thông minh qua GitHub Releases, giúp cập nhật firmware dễ dàng chỉ bằng một click trên web.
 4. Đạt được mức tiết kiệm nước từ 30–40% so với tưới thủ công và đảm bảo cây luôn được tưới đúng lượng, đúng thời điểm.
 5. Tạo nền tảng có thể mở rộng thành hệ thống tưới nhiều vùng, nhiều cây trong tương lai.

1.4. Đối tượng và phạm vi nghiên cứu

Đối tượng phục vụ:

- Hộ gia đình trồng cây cảnh, rau sạch tại nhà, ban công, sân thượng.
- Người yêu thích tự làm (DIY), sinh viên, maker muốn sở hữu hệ thống IoT hoàn toàn tự chủ.
- Các vườn rau nhỏ, nhà kính mini cần giải pháp tưới tự động chi phí thấp.

Phạm vi triển khai (giai đoạn hiện tại – giữa kỳ):

- Quy mô: 1–3 chậu cây/vùng tưới.
- Phần cứng: ESP32 Devkit V1, cảm biến độ ẩm đất điện dung, DHT22 (tùy chọn), relay 1 kênh, bơm mini 5–12V.
- Giao diện: Web dashboard tự xây dựng
- OTA: cập nhật firmware trực tiếp từ GitHub Releases.
- Môi trường hoạt động: trong nhà và ngoài trời có mái che, nhiệt độ 10°C – 50°C.

Không thuộc phạm vi hiện tại:

- Tưới phun sương hoặc tưới nhỏ giọt quy mô lớn.
- Tích hợp AI, dự báo thời tiết.

1.5. Phương pháp nghiên cứu

Dự án được thực hiện theo các phương pháp sau:

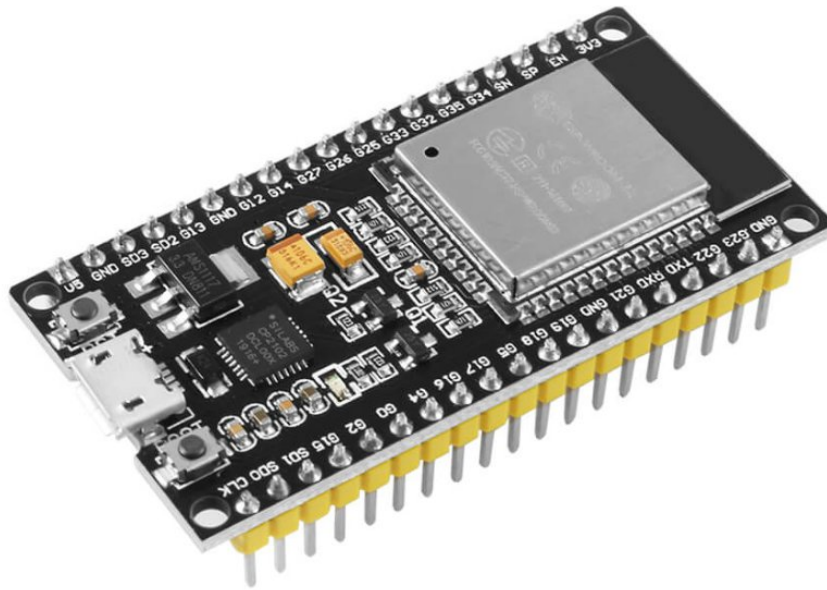
1. Nghiên cứu tài liệu về ESP32, ESPAsyncWebServer, WebSocket, HTTP OTA, GitHub Releases.
2. Thiết kế và lắp ráp phần cứng, vẽ sơ đồ nguyên lý.
3. Lập trình firmware bằng Arduino IDE/PlatformIO, sử dụng các thư viện:
 - AsyncTCP, ESPAsyncWebServer, AsyncElegantOTA
 - ArduinoJson, WebSocketsServer
 - HTTPClient để lấy file .bin và version.txt từ GitHub
4. Thiết kế giao diện web responsive (HTML5, CSS3, JavaScript ES6, Chart.js).

5. Triển khai GitHub Repository với cấu trúc rõ ràng, tạo Releases chứa file .bin và version.txt.
6. Kiểm thử thực tế: độ chính xác cảm biến, thời gian phản hồi web (< 1 giây), độ ổn định OTA, khả năng hoạt động liên tục 24/7.
7. Thu thập dữ liệu thực tế, đo lường nước tiết kiệm và cải tiến hệ thống.

CHƯƠNG 2. Nền tảng lý thuyết

2.1. Module ESP32

2.1.1. Giới thiệu chung (<https://iotzone.vn/esp32/esp32-co-ban/gioi-thieu-esp32-la-gi/>)



ESP32 là một dòng chip vi điều khiển được phát triển bởi Espressif, với nhiều đặc điểm ưu việt:

- **Giá rẻ:** So với các dòng vi điều khiển khác, ESP32 có giá thành phải chăng hơn rất nhiều, giúp tất cả những ai đam mê công nghệ có thể dễ dàng tiếp cận nó
- **Lượng điện tiêu thụ thấp:** So với các chip điều khiển khác, ESP32 tiêu thụ rất ít năng lượng. Dòng chip này cũng hỗ trợ các trạng thái tiết kiệm năng lượng như Deep Sleep để tiết kiệm điện
- **Có thể kết nối Wi-Fi:** Có thể dễ dàng kết nối ESP32 với mạng Wi-Fi để truy cập vào Internet (chế độ trạm – Station mode) hoặc tạo một mạng WiFi cho riêng nó (chế độ điểm truy cập – Access point) để các thiết bị khác có thể kết nối với nó. Chế độ Access point thường được dùng trong các dự án IoT hoặc tự động hóa trong Smart Home, trong đó có thể cho phép nhiều thiết bị liên lạc và trao đổi thông tin với nhau thông qua WiFi của chúng
- **Hỗ trợ Bluetooth:** ESP32 hỗ trợ cả 2 chế độ: Bluetooth Classic và Bluetooth Low Energy (BLE) – một công cụ rất hữu ích cho những ứng dụng IoT
- **Lỗi kép:** Đa số các dòng chip ESP32 hiện nay đều có lỗi kép, chúng đi kèm với 2 bộ vi xử lý Xtensa 32-bit LX6: lỗi 0 và lỗi 1
- **Đa dạng thiết bị ngoại vi:** ESP32 hỗ trợ nhiều loại thiết bị ngoại vi đầu vào (đọc dữ liệu từ bên ngoài) và đầu ra (gửi lệnh/tín hiệu ra bên ngoài) như cảm ứng điện dung, I2C, DAC, PWM, UART, SPI,... để tự do làm các dự án điện tử mà mình thích

- **Tương thích với Arduino và MicroPython:** ESP32 có thể được lập trình bằng ngôn ngữ lập trình phổ biến Arduino và MicroPython (phiên bản rút gọn của Python 3, phù hợp cho các bộ vi điều khiển và hệ thống nhúng)

Thông số kỹ thuật

1. Kết nối không dây

- **WiFi:** Tốc độ dữ liệu lên đến 150Mbps với HT40
- **Bluetooth:** Hỗ trợ BLE (Bluetooth Low Energy – Bluetooth năng lượng thấp) và Bluetooth Classic
- **Bộ xử lý:** Chip vi xử lý LX6 32-bit Tensilica Xtensa Dual-Core, hoạt động ở tốc độ 160MHz hoặc 240MHz

2. Bộ nhớ

- **ROM:** 448 KB (phục vụ cho việc khởi động và chạy các chức năng cốt lõi)
- **SRAM:** 520 KB (dùng cho dữ liệu và hướng dẫn)
- **RTC fast SRAM:** 8 KB (dùng để lưu trữ dữ liệu và CPU chính trong khi RTC Boot ở chế độ Deep Sleep)
- **RTC slow SRAM :** 8KB (dùng để truy cập bộ đồng xử lý (co-processor) trong chế độ Deep Sleep)
- **eFuse:** 1 Kbit, trong đó:
 - 256 bit: phục vụ cho hệ thống (địa chỉ MAC và cấu hình chip)
 - 768 bit còn lại: được dành riêng cho các ứng dụng của người dùng, ví dụ như Flash-Encryption và Chip-ID
- **Embedded flash:** Flash được kết nối thông qua IO16, IO17, SD_CMD, SD_CLK, SD_DATA_0 and SD_DATA_1 on ESP32-D2WD and ESP32-PICO-D4. Cụ thể:
 - 0 MiB (ESP32-D0WDQ6, ESP32-D0WD, and ESP32-S0WD chips)
 - 2 MiB (ESP32-D2WD chip)
 - 4 MiB (ESP32-PICO-D4 SiP module)

3. Công suất

ESP32 cho phép vẫn có thể sử dụng bộ chuyển đổi ADC, kể cả khi đang trong chế độ Deep Sleep.

4. Hỗ trợ các thiết bị ngoại vi đầu ra / đầu vào

- Giao diện ngoại vi với DMA bao gồm cảm ứng điện dung
- ADC (Bộ chuyển đổi Analog thành Digital, tên đầy đủ là Analog to Digital Converter)
- DACs (Bộ chuyển đổi Digital thành Analog, tên đầy đủ là Digital to Analog Converter)
- I2C (Inter Integrated Circuit)

- UART (Universal Asynchronous Receiver/Transmitter)
- SPI (Serial Peripheral Interface)
- I2S (Integrated Interchip Sound)
- RMII (Reduced Media-Independent Interface)
- PWM (Pulse-Width Modulation)

5. Tính bảo mật

Bộ tăng tốc phần cứng dành cho SSL/TLS và AES

2.1.2. Nguyên lý hoạt động

Nguyên lý hoạt động của ESP32 dựa trên việc kết hợp bộ xử lý lõi kép mạnh mẽ, tích hợp Wi-Fi và Bluetooth, cho phép xử lý dữ liệu, kết nối mạng và giao tiếp với các thiết bị ngoại vi khác thông qua nhiều giao thức như ADC, SPI, I2C, UART. ESP32 nhận tín hiệu từ các cảm biến (như cảm biến điện dung, nhiệt độ) và xử lý chúng, sau đó thực hiện các tác vụ như kết nối mạng, gửi dữ liệu lên đám mây hoặc điều khiển các thiết bị khác

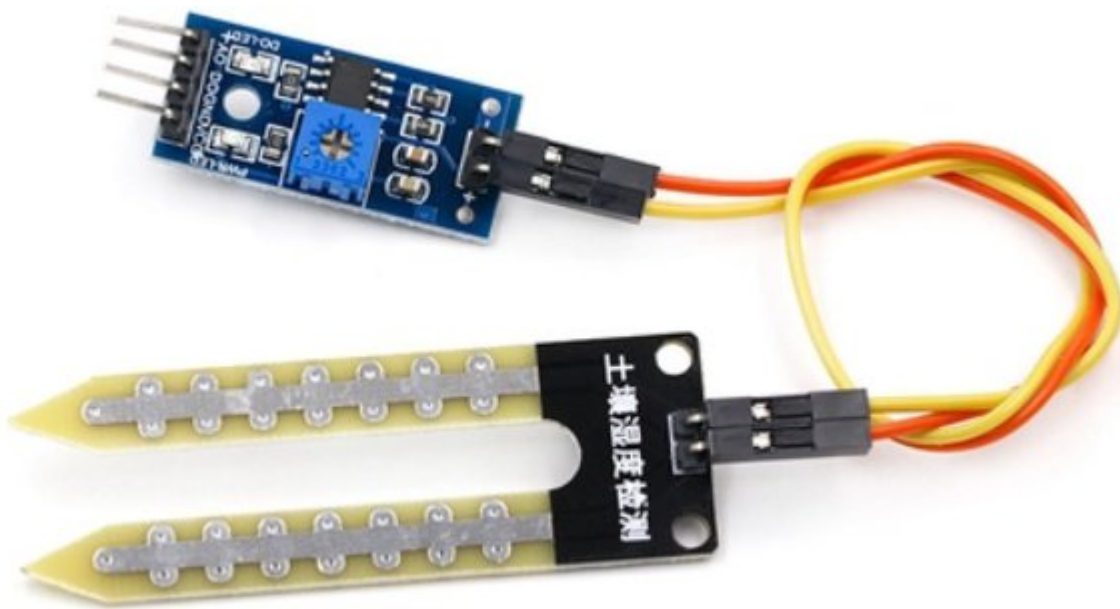
2.1.3. Sự khác biệt so với ESP8266 (<https://iotzone.vn/esp32/esp32-co-ban/gioi-thieu-esp32-la-gi/>)

- ESP32 có tốc độ nhanh hơn ESP8266
- ESP32 đi kèm với nhiều GPIO hơn, mang đến nhiều chức năng đa dạng
- ESP32 hỗ trợ các chuyển đổi Analog trên 18 kênh (chân kích hoạt Analog), trong khi đó ESP8266 chỉ có một chân ADC 10 bit
- ESP32 hỗ trợ Bluetooth còn ESP8266 thì không có
- ESP32 cỡ lõi kép (hầu hết các sản phẩm), còn ESP8266 chỉ dùng lõi đơn
- ESP32 đắt hơn một chút so với ESP8266

2.2. Các cảm biến sử dụng

2.2.1. Cảm biến độ ẩm đất điện dung (<https://imaker.vn/cam-bien-do-am-dat>)

2.2.1.1. Giới thiệu chung



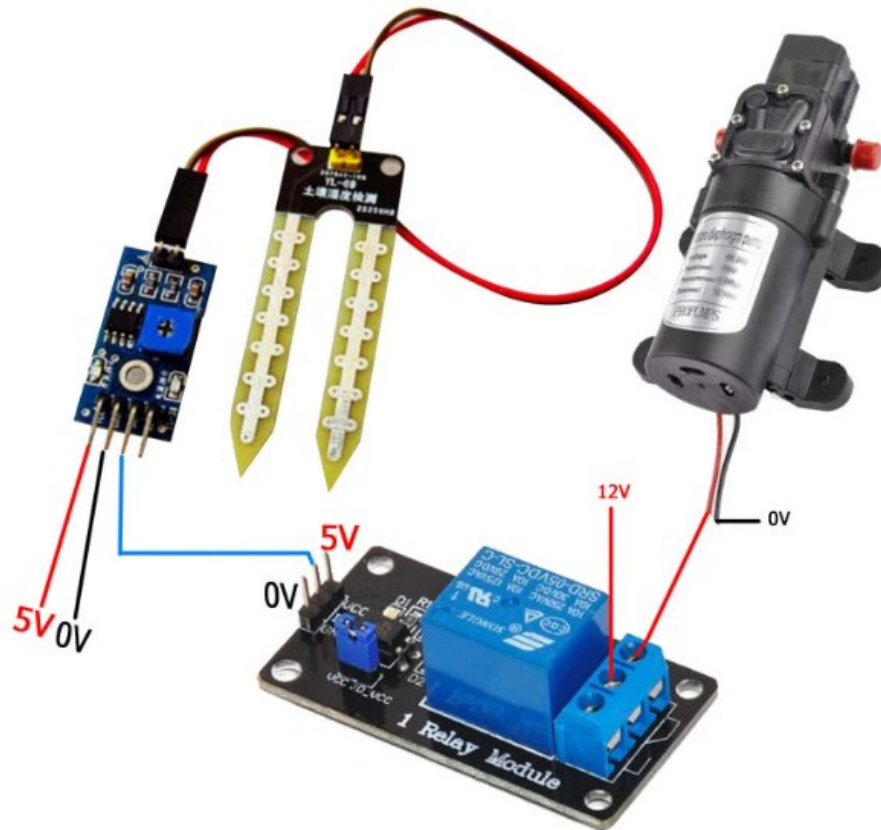
Cảm biến độ ẩm đất Soil Moisture Sensor là thiết bị điện tử được sử dụng để đo lượng nước trong đất, giúp theo dõi tình trạng độ ẩm của đất và hỗ trợ các ứng dụng nông nghiệp, trồng trọt, và hệ thống tưới tiêu tự động.

Thông số kỹ thuật:

- Điện áp hoạt động: 3.3V-5V
- Kích thước PCB: 3cm * 1.6cm
- Led đỏ báo nguồn vào, Led xanh báo độ ẩm.
- IC so sánh : LM393
- VCC: 3.3V-5V
- GND: 0V
- DO: Đầu ra tín hiệu số (0 và 1)
- AO: Đầu ra Analog (Tín hiệu tương tự)

2.2.1.2. Nguyên lý hoạt động

Ví dụ về một hệ thống tưới nước



- Khi đất khô, module cảm biến đưa ra chân D0 mức 1. Module role không hoạt động nên bơm hoạt động.
- Khi nước đủ ẩm, chân D0 xuống mức 0 , modul Role hoạt động nên bơm không hoạt động.

2.2.2. Cảm biến nhiệt độ, độ ẩm không khí DHT11

(<https://www.studocu.vn/vn/document/vnu-university-of-engineering-and-technology/nguyen-ly-ky-thuat-dien-tu/dht11-thank-you/111744380>)

2.2.2.1. Giới thiệu chung



DHT11 là một cảm biến đo nhiệt độ và độ ẩm kỹ thuật số đơn giản và giá thành rất rẻ, được sử dụng phổ biến trong các ứng dụng đo lường môi trường cơ bản. Cảm biến này sử dụng một cảm biến độ ẩm điện dung và một điện trở nhiệt (thermistor) để đo lường độ ẩm và nhiệt độ không khí. Cảm biến này cung cấp dữ liệu qua tín hiệu kỹ thuật số, vì vậy không cần sử dụng các chân đầu vào analog, điều này giúp giảm độ phức tạp trong việc lập trình và kết nối với các vi điều khiển.

Thông số kỹ thuật

- Nguồn: 3 -> 5 VDC.
- Dòng sử dụng: 2.5mA max (khi truyền dữ liệu).
- Đo tốt ở độ ẩm 20 to 70%RH với sai số 5%.
- Đo tốt ở nhiệt độ 0 to 50°C sai số $\pm 2^\circ\text{C}$.
- Tần số lấy mẫu tối đa 1Hz (1 giây 1 lần)
- Kích thước 15mm x 12mm x 5.5mm.
- 4 chân, khoảng cách chân 0.1".

2.2.2.2. Nguyên lý hoạt động

Cảm biến DHT11 bao gồm hai thành phần chính để thực hiện chức năng đo nhiệt độ và độ ẩm: phần đo độ ẩm và phần đo nhiệt độ

Đ. Phần Đo Độ Ẩm (Capacitive Humidity Sensor) sử dụng điện cực và một lớp

- Phần cảm biến này sử dụng một tụ điện với hai điện cực và một lớp
 - Chất nền giữ ẩm (dielectric) đặt ở giữa. Chất nền này là một vật liệu có khả năng giữ ẩm, và khi độ ẩm thay đổi trong môi trường không khí, chất

nền này sẽ thay đổi khả năng lưu trữ nước

- Thay đổi điện dung: Khi độ ẩm trong không khí thay đổi, độ ẩm này sẽ tác động đến điện môi giữa các điện cực của tụ điện, làm thay đổi điện dung của tụ. Sự thay đổi này có thể được tính toán và chuyển đổi thành tín hiệu số thông qua vi mạch IC bên trong cảm biến
- IC xử lý: IC của cảm biến sẽ đo lường sự thay đổi trong điện dung và xử lý thông tin này để chuyển đổi thành giá trị độ ẩm, sau đó xuất ra dưới dạng tín hiệu kỹ thuật số.

B. Phân đo nhiệt độ

- Thermistor (điện trở nhiệt)
 - Cảm biến DHT11 sử dụng một thermistor có hệ số nhiệt âm (NTC - Negative Temperature Coefficient). Điều này có nghĩa là điện trở của thermistor sẽ giảm khi nhiệt độ tăng.
 - Cấu tạo: Thermistor của DHT11 thường được làm từ các vật liệu như gốm bán dẫn hoặc polymer, giúp tăng độ nhạy của cảm biến với các thay đổi nhiệt độ nhỏ
 - Lý do sử dụng thermistor: Các thermistor này có đặc tính giúp thay đổi điện trở một cách rõ rệt với mỗi biến động nhiệt độ nhỏ, điều này giúp cảm biến DHT11 đạt được độ chính xác cao trong việc đo nhiệt độ.

2.2.3. Cảm biến ánh sáng BH1750

2.2.3.1. Giới thiệu chung (<https://dientuviet.com/cam-bien-cuong-do-anh-sang-bh1750/>)

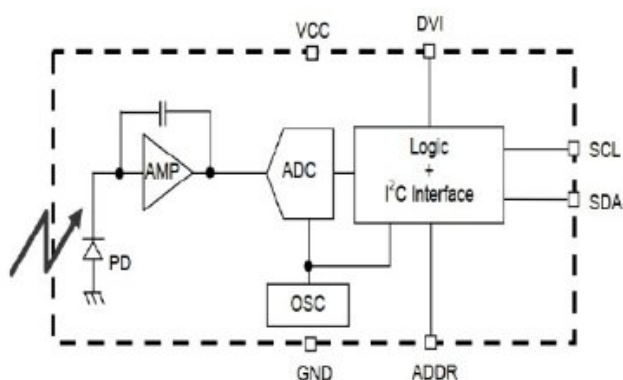


BH1750 là một IC cảm biến số 16 bit dùng để đo cường độ ánh sáng môi trường xung quanh giao tiếp qua giao thức I2C. IC này là thích hợp nhất để lấy dữ liệu ánh sáng xung quanh để điều chỉnh công suất đèn nền của màn hình LCD và bàn phím của điện thoại di động. Cảm biến có thể đo cường độ ánh sáng trong phạm vi rộng với độ phân giải cao (1 lux – 65535 lux)

Thông số kỹ thuật

- Điện áp hoạt động: 2,4V – 3,6VDC
- Chuẩn giao tiếp: I2C
- Dải đo ánh sáng: 1 – 65535 lx
- Đặc điểm độ nhạy phổ: Độ nhạy cực đại với bước sóng 560nm
- Khả năng phát hiện các nguồn sáng như: đèn sợi đốt, đèn huỳnh quang, đèn LED trắng, đèn huỳnh quang,..
- Kích thước: 2,6 x 2,8 cm

2.2.3.2. Nguyên lý hoạt động (<https://www.studocu.vn/vn/document/truong-dai-hoc-cong-nghe-tphcm/administration-bussiness/nhom5-bao-cao-do-an/108260571>)

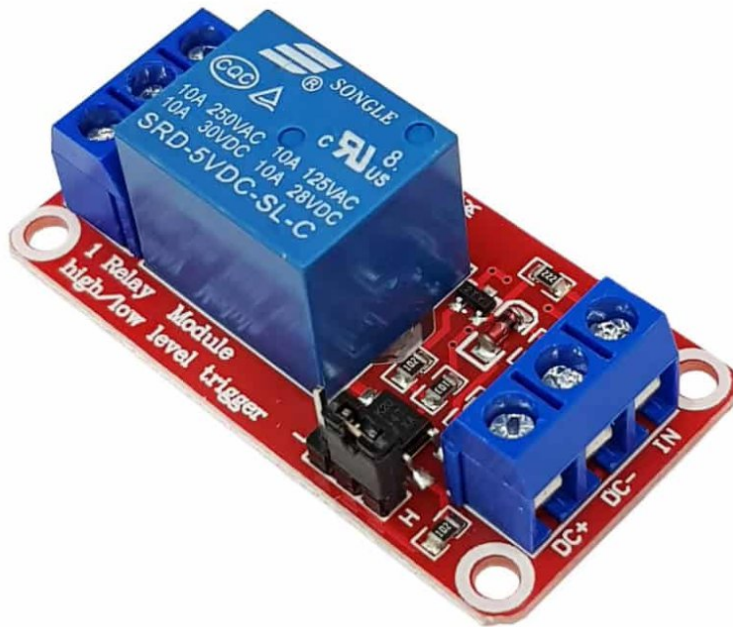


- PD : Diode quang là phần tử cảm biến trong hệ thống, đóng vai trò chuyển đổi tín hiệu quang sang tín hiệu điện
- AMP : Là bộ khuếch đại dùng để tăng cường độ tín hiệu nhận được từ cảm biến
- ADC: Là bộ chuyển đổi tương tự sang số, dùng để chuyển đổi tương tự thu được từ cảm biến sau khi đã qua bộ khuếch đại , sang dạng tín hiệu số
- Logic + I2C Interface: Là khối xử lý tín hiệu số và truyền thông giao tiếp I2c
- OSC: Là mạch dao động để cấp dao động cho các mạch hoạt động

2.3. Thiết bị chấp hành

2.3.1. Relay module (<https://plctech.com.vn/relay-la-gi/>)

2.3.1.1. Giới thiệu chung



Relay là một công tắc điện từ hoạt động nhờ vào một dòng điện nhỏ, có khả năng bật hoặc tắt một dòng điện lớn hơn nhiều lần. Trái tim của relay là cuộn dây điện (hay còn gọi là nam châm điện) – khi có dòng điện chạy qua, cuộn dây sẽ tạo ra một từ trường, khiến nó trở thành một nam châm tạm thời. Từ đó, relay có thể đóng mở các tiếp điểm điện, giúp điều khiển dòng điện lớn hơn.

Cấu tạo của relay

Relay bao gồm 3 phần chính, mỗi phần đảm nhiệm một chức năng riêng:

- Khối tiếp nhận (cơ cấu tiếp nhận): Nhận tín hiệu đầu vào và chuyển đổi nó thành tín hiệu thích hợp để truyền đến khối trung gian.
- Khối trung gian (cơ cấu trung gian): Tiếp nhận tín hiệu từ khối tiếp nhận và chuyển đổi nó thành tín hiệu điều khiển cho relay.
- Khối chấp hành (cơ cấu chấp hành): Thực hiện tác động vào mạch điều khiển, giúp relay đóng hoặc mở các tiếp điểm điện.

Thông số kỹ thuật

Hiệu điện thế kích tối ưu:

- Đây là yếu tố quyết định việc relay có hoạt động đúng hay không. Ví dụ, nếu cần điều khiển một bóng đèn 220V từ một cảm biến ánh sáng hoạt động ở mức 5 – 12V, sẽ cần một module relay có hiệu điện thế kích là 5V hoặc 12V.

Hiệu điện thế và cường độ dòng điện tối đa:

- Thông số này cho biết mức dòng điện và điện áp tối đa mà relay có thể chịu đựng. Thông thường, chúng ta có thể tìm thấy thông tin này được in sẵn trên thiết bị.

2.3.1.2. Nguyên lý hoạt động

- Điện được cấp cho cuộn dây, tạo ra từ trường
- Từ trường tác động lên phần ứng, tạo ra lực hút

- Phản ứng di chuyển, đóng hoặc mở các tiếp điểm điện
- Các tiếp điểm chuyển mạch dòng điện đến các thiết bị như động cơ, bóng đèn, v.v.
- Khi điện áp bị loại bỏ, từ trường biến mất và các tiếp điểm trở lại vị trí ban đầu.

2.3.2. Máy bơm nước (<https://hshop.vn/dong-co-bom-chim-mini-5vdc>)

2.3.2.1. Giới thiệu chung



Động cơ bơm chìm Mini Water Pump 5VDC có kích thước rất nhỏ gọn, sử dụng điện áp 3~5VDC, vì thuộc dạng bơm chìm nên động cơ có khả năng chống nước và hoạt động khi ngâm chìm trong nước, ứng dụng để bơm nước, dung dịch trong các thiết kế nhỏ, mô hình tưới cây, hồ cá,..., động cơ có hai phiên bản là không kèm cáp USB và kèm cáp USB

Thông số kỹ thuật

- Điện áp sử dụng: 3~5VDC.
- Dòng điện sử dụng: 100~200mA.
- Lưu lượng bơm: 1.2~1.6L / 1 phút.
- Đường kính ngoài ống dẫn: 7.5mm
- Kích thước: 34 x 43 mm
- Trọng lượng: 28g

2.3.2.2. Nguyên lý hoạt động

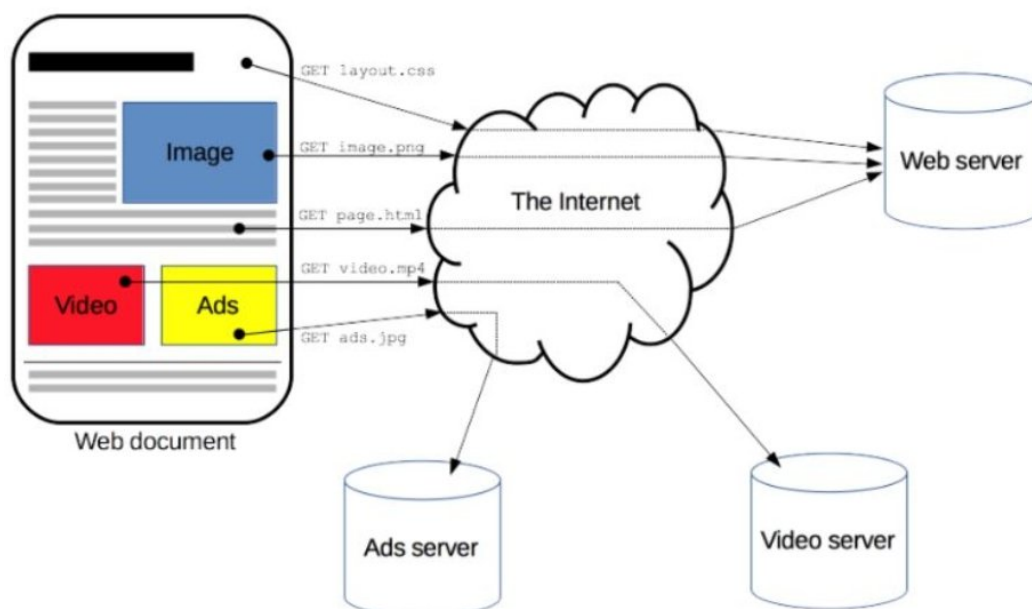
Máy bơm mini IoT hoạt động bằng cách kết hợp nguyên lý hoạt động của máy bơm nước truyền thống và công nghệ Internet of Things (IoT) để điều khiển từ xa. Máy bơm sẽ hoạt động khi áp suất nước giảm dưới ngưỡng cài đặt, và ngừng lại khi áp suất đạt mức mong

muốn, với tín hiệu từ cảm biến được xử lý qua mạng internet để ra lệnh cho động cơ máy bơm thông qua một bộ điều khiển (như Raspberry Pi) và relay

2.4. Các giao thức truyền thông

2.4.1. Giao thức HTTP/HTTPS

2.4.1.1. Giao thức HTTP (<https://fptshop.com.vn/tin-tuc/danh-gia/http-la-gi-166491> , <https://vinahost.vn/http-la-gi/>)



HTTP là viết tắt của HyperText Transfer Protocol, là một giao thức truyền tải siêu văn bản, giúp cho các máy tính có thể giao tiếp với nhau qua mạng. HTTP là nền tảng của World Wide Web kết nối giữa máy chủ (server) và máy khách (client) trong cùng một hệ thống mạng. Điều này cho phép chúng ta truy cập vào các trang web, tải xuống các tệp tin, xem các hình ảnh, video... Ban đầu khi được thiết kế vào những năm 90, HTTP là một giao thức linh hoạt có khả năng mở rộng theo thời gian. Giao thức này hoạt động trên nền tảng TCP/IP và thường được truyền qua kết nối mã hóa TLS để bảo vệ dữ liệu. Mặc dù về lý thuyết, bất kỳ giao thức truyền tải đáng tin cậy nào cũng có thể được áp dụng

Với khả năng mở rộng đa dạng, HTTP không chỉ được sử dụng để tải các tài liệu siêu văn bản, mà còn để truyền tải hình ảnh, video và thậm chí để đăng tải nội dung lên máy chủ, chẳng hạn như kết quả của các biểu mẫu HTML. Ngoài ra, HTTP cũng có thể sử dụng để tải lên các phần của trang web, giúp cập nhật nội dung theo yêu cầu.

HTTP được sử dụng rộng rãi trong việc truy cập các trang web trên Internet. Khi nhập một địa chỉ web vào trình duyệt, trình duyệt sẽ gửi một yêu cầu HTTP đến máy chủ web, và máy chủ web sẽ trả về một phản hồi HTTP chứa mã HTML của trang web đó. Trình duyệt sẽ đọc mã HTML và hiển thị trang web cho xem. Có thể thấy các yêu cầu và phản hồi HTTP bằng cách sử dụng công cụ kiểm tra Đa mạng (network inspector) của trình duyệt

Đặc điểm của HTTP

- Tính đơn giản của HTTP

Giao thức HTTP được thiết kế với sự đơn giản và thân thiện đối với người đọc, ngay cả khi có sự phức tạp được tích hợp trong HTTP/2 thông qua việc đóng gói các HTTP message thành các frame. Bằng cách này, việc đọc và hiểu các thông điệp HTTP trở nên dễ dàng hơn, cung cấp khả năng kiểm thử cao hơn cho các nhà phát triển và giảm bớt độ phức tạp đối với người mới sử dụng.

- **Khả năng mở rộng của HTTP**

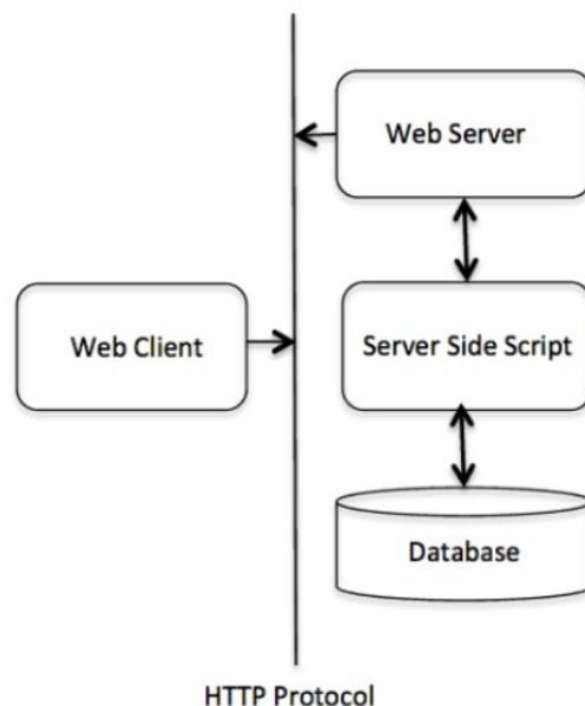
Tính năng header HTTP đã được giới thiệu từ HTTP/1.0, đã tạo điều kiện thuận lợi cho việc mở rộng và thử nghiệm giao thức. Các chức năng mới có thể dễ dàng được giới thiệu thông qua thỏa thuận đơn giản giữa client và máy chủ về ý nghĩa của một header mới.

- **Tính stateless nhưng không phải sessionless của HTTP**

Trong khi HTTP được xem là stateless, không giữ liên kết giữa hai yêu cầu được thực hiện liên tiếp trên cùng một kết nối, điều này có thể tạo ra thách thức khi người dùng cố gắng tương tác mạch lạc với các trang cụ thể, như việc sử dụng giỏ hàng trên các trang thương mại điện tử. Mặc dù vậy, một giải pháp tinh tế là sử dụng cookie HTTP, nhờ vào khả năng mở rộng của header. Cookie HTTP có thể được tích hợp vào quy trình hoạt động, cho phép tạo ra các phiên trạng thái trên mỗi yêu cầu HTTP, từ đó chia sẻ cùng một ngữ cảnh hoặc trạng thái.

Cấu trúc cơ bản của HTTP

HTTP là giao thức hoạt động dựa trên mô hình **yêu cầu-phản hồi** giữa máy khách (client) và máy chủ (server). Khi người dùng nhập một URL, trình duyệt (client) gửi yêu cầu tới máy chủ chứa nội dung đó và máy chủ sẽ phản hồi với dữ liệu được yêu cầu.



Phương thức HTTP

HTTP hỗ trợ nhiều phương thức yêu cầu khác nhau, mỗi phương thức có mục đích riêng trong việc xử lý tài nguyên trên máy chủ:

- GET: Yêu cầu truy xuất tài nguyên từ máy chủ (ví dụ: tải một trang web).
- POST: Gửi dữ liệu đến máy chủ để tạo mới hoặc cập nhật tài nguyên.
- PUT: Tương tự POST, nhưng thường dùng để cập nhật toàn bộ tài nguyên.
- DELETE: Xóa tài nguyên trên máy chủ.
- HEAD: Tương tự GET nhưng không trả về phần nội dung chính của tài nguyên, chỉ trả về các tiêu đề.
- PATCH: Cập nhật một phần tài nguyên (khác với PUT cập nhật toàn bộ).
- OPTIONS: Lấy thông tin về các phương thức HTTP mà máy chủ hỗ trợ cho tài nguyên cụ thể.

2.4.2. Giao thức HTTPS (<https://mona.media/https-la-gi/>)

HTTPS là viết tắt của chữ Hypertext Transfer Protocol Secure nghĩa là giao thức truyền tải siêu văn bản an toàn. Nguồn gốc xuất phát chính là giao thức HTTP, tuy nhiên nó được tích hợp thêm chứng chỉ bảo mật SSL mục đích nhằm mã hóa các thông điệp giao tiếp để tăng cường tính bảo mật tối ưu. Có thể hiểu theo nghĩa khác như, HTTPS chính là phiên bản khác của HTTP nhưng an toàn và bảo mật hơn.

Hoạt động của HTTPS nhìn chung cũng tương tự như HTTP, nhưng mặt khác nó được bổ sung đầy đủ thêm chứng chỉ SSL (Secure Sockets Layer – tầng ổ bảo mật) hoặc TLS (Transport Layer Security – bảo mật tầng truyền tải). Hiện tại, đây là những tiêu chuẩn bảo mật hàng đầu áp dụng cho hàng triệu website trên toàn thế giới

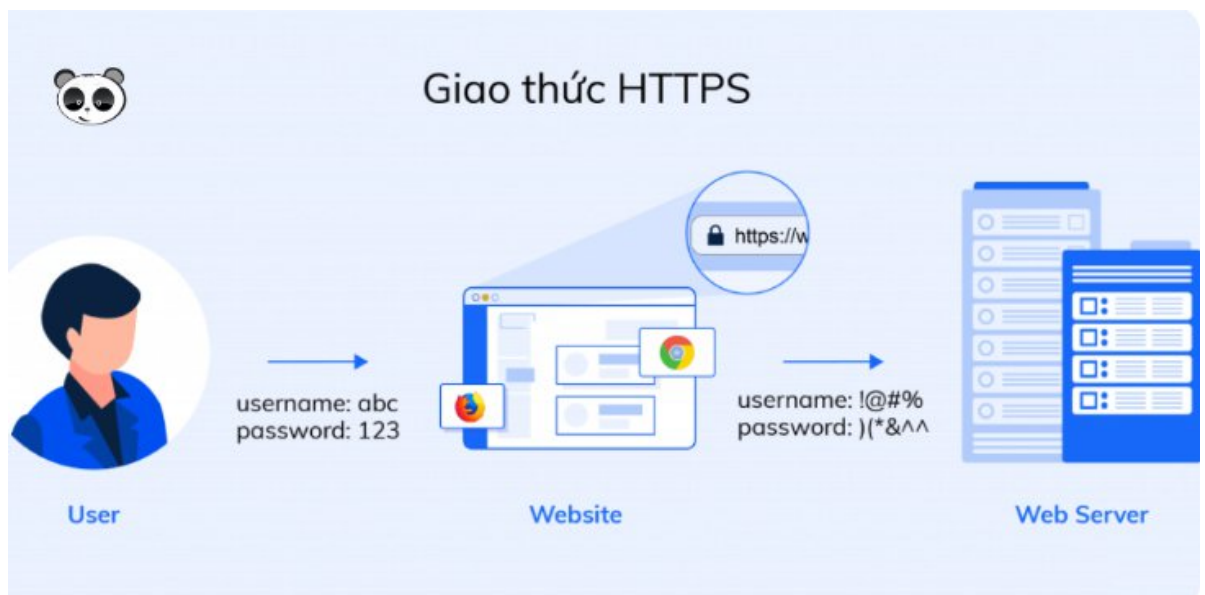
Cả SSL và TLS đều đang sử dụng hệ thống PKI (Public Key Infrastructure - hạ tầng khóa công khai) với cấu trúc không đối xứng. Hệ thống này áp dụng hai “khóa” để mã hóa thông tin liên lạc, bao gồm “khóa công khai” (public key) và “khóa riêng” (private key). Đối với mã khóa công khai, mọi thứ đều chỉ có thể được giải mã bởi khóa riêng và ngược lại. Tất cả các tiêu chuẩn này sẽ đảm bảo các nội dung được mã hóa trước khi truyền đi và giải mã khi nhận. Điều này sẽ giúp việc bảo vệ mã được tốt hơn vì khiến hacker dù có chen ngang lấy thông tin cũng không thể “hiểu” được thông tin đó.

Quá trình giao tiếp giữa client và server thông qua giao thức HTTPS

- Client sẽ gửi request cho một secure page (có URL bắt đầu với https://)
- Sau đó Server gửi phản hồi lại cho client certificate của nó.
- Tiếp đến Client (web browser) tiến hành kiểm chứng xác thực certificate này bằng cách kiểm tra (verify) dựa trên tính hợp lệ của chữ ký số của CA được kèm theo certificate. Giả sử trường hợp ,certificate đã được xác thực và còn hạn sử dụng hoặc client vẫn muốn truy cập dù cho Web browser đã cảnh báo trước rằng certificate này có thể không đáng tin cậy (do là dạng self-signed SSL certificate hoặc certificate hết

hiệu lực hay thông tin trong certificate cập nhật không đúng) thì lúc đó mới xảy ra diễn biến nằm ở bước 4 sau.

- Client ngẫu nhiên tự tạo một symmetric encryption key (hay còn gọi là session key), rồi sử dụng public key (được lấy trong certificate) để mã hóa session key này và sau đó gửi về cho server.
- Server sẽ sử dụng private key này (tương ứng với public key trong certificate ở trên) để giải mã ra session key như ở trên.
- Cuối cùng, cả server và client đều sử dụng session key đó để mã hóa/giải mã các thông điệp trong suốt quá trình phiên truyền thông.



2.5. Cơ chế cập nhật firmware từ xa (OTA)

2.5.1. Khái niệm OTA (<https://www.quora.com/What-is-an-OTA-update>)

Cập nhật OTA (Over-The-Air) là phương pháp cung cấp các bản cập nhật phần mềm, thay đổi cấu hình hoặc bản vá firmware không dây cho thiết bị mà không cần phương tiện vật lý hoặc cài đặt thủ công. Phương pháp này được sử dụng rộng rãi trên điện thoại thông minh, máy tính bảng, thiết bị IoT, ô tô, đầu thu kỹ thuật số và hệ thống nhúng.

Các thành phần chính và quy trình làm việc

- Máy chủ phân phối: chuẩn bị các gói cập nhật đã ký và quản lý các chính sách triển khai (phát hành theo giai đoạn, kênh A/B, nhắm mục tiêu thiết bị).
- Kênh phân phối: sử dụng mạng di động, Wi-Fi hoặc các phương thức truyền tải mạng khác để đẩy siêu dữ liệu và tải trọng cập nhật.
- Trình cập nhật trên thiết bị: kiểm tra bản cập nhật, tải xuống gói, xác minh chữ ký số, xác thực tính toàn vẹn và sắp xếp cài đặt.
- Áp dụng cơ chế: có thể sử dụng thay thế tại chỗ, phân vùng A/B nguyên tử (khả năng khôi phục liền mạch) hoặc cài đặt theo giai đoạn với chế độ phục hồi.

- Báo cáo và khôi phục: thiết bị báo cáo trạng thái; máy chủ có thể dừng triển khai hoặc kích hoạt khôi phục đối với các bản dựng bị lỗi.

Tầm quan trọng của OTA

- Bảo vắ bảo mật nhanh chóng và cung cấp tính năng ở quy mô lớn.
- Giảm chi phí và hậu cần so với dịch vụ thực tế.
- Mô hình phân phối liên tục cho phép chu kỳ cải tiến sản phẩm nhanh hơn.

2.5.2. Quy trình cập nhật qua github release

Bước 1: Tạo firmware ESP32

- Tạo file firmware.bin

Bước 2: Tạo github realese

- Lên GitHub → vào repo
- Chọn Releases → Draft a new release
- Tạo tag ví dụ: v1.0.1
- Upload file firmware.bin
- Publish

Bước 3: ESP32 kiểm tra phiên bản mới

ESP32 truy cập tệp version.json trên github để so sánh server_version và current_version

Cập nhật nếu server_version > current_version thì thiết bị tải tệp firmware.bin từ đường dẫn trong tệp json

Bước 4: cập nhật

Sau khi tải xong, ESP32 tự động flase firmware mới và khởi động lại

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU CHỨC NĂNG

3.1. Chức năng giám sát môi trường

Hệ thống phải thu thập và hiển thị realtime các thông số môi trường quan trọng để người dùng theo dõi tình trạng cây trồng từ xa qua trình duyệt web (điện thoại/máy tính).

Các thông số cần giám sát:

- Độ ẩm đất (%)
- Nhiệt độ không khí (°C)
- Độ ẩm không khí (%)
- Cường độ ánh sáng (lux) – tùy chọn
- Trạng thái máy bơm (ON/OFF) và thời gian hoạt động gần nhất

Yêu cầu cụ thể:

- Dữ liệu được cập nhật liên tục lên giao diện web qua giao thức **WebSocket** với tần suất ≤ 10 giây/lần.
- Dữ liệu lịch sử được lưu trong bộ nhớ SPIFFS của ESP32 hiển thị dưới dạng biểu đồ (Chart.js) trên web.
- Giao diện web hiển thị realtime bằng gauge, số liệu và đồ thị đường.

3.2. Chức năng điều khiển tưới (Thủ công/Tự động)

Hệ thống hỗ trợ 2 chế độ tưới:

a. Chế độ tự động (Auto)

- ESP32 tự đọc giá trị độ ẩm đất định kỳ.
- Khi độ ẩm đất < ngưỡng dưới $\rightarrow$ tự động bật relay $\rightarrow$ bật máy bơm.
- Khi độ ẩm đất $\geq$ ngưỡng trên $\rightarrow$ tự động tắt máy bơm.
- Có thể cài đặt thêm thời gian tưới tối đa (phòng trường hợp cảm biến lỗi).

b. Chế độ thủ công (Manual)

- Người dùng có thể bật/tắt máy bơm bất kỳ lúc nào thông qua nút bấm trên giao diện web.
- Lệnh được gửi qua WebSocket $\rightarrow$ ESP32 nhận và thực thi ngay lập tức (độ trễ < 1 giây).
- Ưu tiên chế độ thủ công cao hơn tự động khi người dùng đang thao tác.

3.3. Chức năng cấu hình ngưỡng

Người dùng có thể tùy chỉnh các thông số hoạt động của hệ thống ngay trên giao diện web mà không cần nạp lại code:

Các ngưỡng có thể cấu hình:

- Ngưỡng độ ẩm đất thấp nhất (ví dụ: 30%)
- Ngưỡng độ ẩm đất cao nhất (ví dụ: 70%)
- Thời gian tưới tối đa mỗi lần (giây)
- Chu kỳ đọc cảm biến (giây)
- Chế độ hoạt động (Auto/Manual)

Cách thức thực hiện:

- Dữ liệu cấu hình được lưu vào **EEPROM** hoặc **Preferences** của ESP32 để không mất khi mất điện.
- Giao diện web cung cấp form nhập liệu + nút “Lưu cấu hình”.
- Sau khi lưu, ESP32 tự động reload các ngưỡng mới.

3.4. Chức năng cập nhật Firmware (Check Version & Update)

Đây là tính năng nổi bật của đề tài – hỗ trợ cập nhật firmware từ xa hoàn toàn qua GitHub mà không cần cáp.

Yêu cầu chi tiết:

- ESP32 định kỳ (hoặc khi người dùng nhấn nút “Check Update” trên web) sẽ gửi HTTP request tới GitHub Releases để lấy file **version.txt**.
- So sánh phiên bản hiện tại (được hard-code hoặc lưu trong EEPROM) với phiên bản mới nhất trên GitHub.
- Nếu có phiên bản mới → hiển thị thông báo trên web + nút “Cập nhật ngay”.
- Khi người dùng nhấn cập nhật → ESP32 tự động tải file **firmware.bin** từ GitHub Release và thực hiện OTA (sử dụng thư viện Update hoặc AsyncElegantOTA).
- Sau khi cập nhật thành công → tự động khởi động lại và hiển thị phiên bản mới trên giao diện web.
- Toàn bộ quá trình được hiển thị progress bar realtime qua WebSocket.

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

4.1. Sơ đồ khối hệ thống

Hệ thống tưới cây thông minh hoạt động dựa trên mô hình vòng kín: Thu thập dữ liệu môi trường -> Xử lý trung tâm -> Ra quyết định điều khiển -> Tương tác người dùng. Sơ đồ khối tổng quát được chia thành 4 khối chức năng chính:

Mô tả sơ đồ khối:

1. Khối cảm biến (Input Block):

- Gồm cảm biến độ ẩm đất (Analog), cảm biến nhiệt độ - độ ẩm không khí DHT11 (Digital), và cảm biến ánh sáng BH1750 (I2C).
- Nhiệm vụ: Thu thập liên tục các thông số vật lý của môi trường trồng cây và chuyển đổi thành tín hiệu điện gửi về khối xử lý trung tâm.

2. Khối xử lý trung tâm (Processing Block):

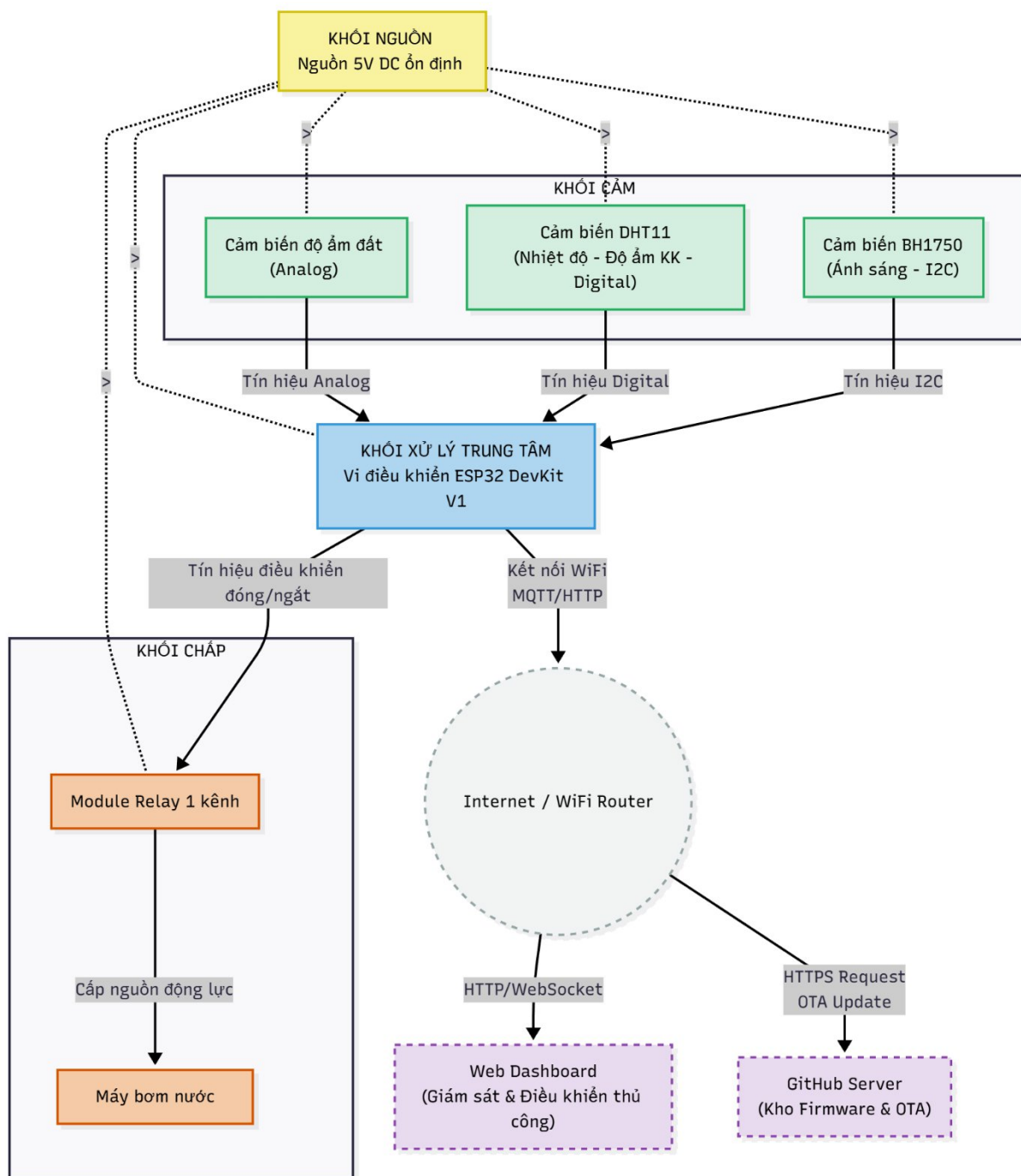
- Sử dụng vi điều khiển **ESP32 DevKit V1**.
- Nhiệm vụ: Nhận tín hiệu từ cảm biến, xử lý dữ liệu (tính toán, so sánh với ngưỡng cài đặt), thực thi thuật toán điều khiển (Auto/Manual), quản lý kết nối WiFi và giao tiếp với Web Server/GitHub.

3. Khối chấp hành (Output Block):

- Gồm Module Relay 1 kênh và Máy bơm nước mini.
- Nhiệm vụ: Nhận lệnh đóng/ngắt từ ESP32 để cấp nguồn hoặc ngắt nguồn cho máy bơm, thực hiện việc tưới nước.

4. Khối giám sát và nguồn (Power & Interface Block):

- **Nguồn:** Cung cấp điện áp 5V ổn định cho toàn bộ hệ thống.
- **Giao diện Web & GitHub:**
 - *Web Dashboard:* Hiển thị thông số realtime, cho phép người dùng cấu hình và điều khiển thủ công.
 - *GitHub:* Chứa kho lưu trữ firmware, phục vụ tính năng cập nhật từ xa (OTA).



4.2 Thiết kế phần cứng (Sơ đồ nguyên lý)

4.2.1. Danh sách phần cứng sử dụng

Hệ thống tưới cây thông minh sử dụng các phần cứng chính sau:

- **ESP32 DevKit V1**

Là vi điều khiển trung tâm, tích hợp WiFi, dùng để đọc dữ liệu cảm biến, xử lý logic tưới, giao tiếp với server/web và thực hiện OTA firmware.

- **Cảm biến độ ẩm đất (Soil Moisture Sensor – loại điện dung)**
Dùng để đo độ ẩm của đất tại vị trí gốc cây. Tín hiệu đầu ra dạng analog được đưa vào chân ADC của ESP32 để đánh giá trạng thái khô/ẩm của đất.
- **Cảm biến ánh sáng BH1750**
Là cảm biến cường độ ánh sáng giao tiếp qua giao thức I2C. Dùng để đo mức độ chiếu sáng môi trường, có thể hỗ trợ quyết định tưới (ví dụ: ưu tiên tưới vào sáng sớm/chiều tối).
- **Cảm biến nhiệt độ, độ ẩm không khí DHT11**
Dùng để đo nhiệt độ và độ ẩm không khí môi trường xung quanh khu vực trồng cây, phục vụ giám sát và hiển thị trên giao diện web.
- **Relay module**
Đóng vai trò công tắc điện tử, cách ly giữa phần điều khiển (ESP32) và phần tải công suất (máy bơm nước). ESP32 chỉ điều khiển mức logic (ON/OFF), relay sẽ đóng/ngắt nguồn cho máy bơm.
- **Máy bơm nước**
Thực hiện việc bơm nước từ nguồn chứa đến khu vực tưới. Máy bơm được cấp nguồn riêng (thông qua relay), không cấp trực tiếp từ ESP32.
- **Nguồn ổn định 2-5 V**
Cung cấp điện áp ổn định cho ESP32 (qua chân VIN hoặc module nguồn) và relay module.

4.2.2. Sơ đồ nguyên lý hệ thống

Hệ thống tưới cây thông minh được thiết kế xoay quanh vi điều khiển ESP32 DevKit V1, đóng vai trò là bộ não trung tâm, đọc dữ liệu từ các cảm biến và điều khiển máy bơm thông qua relay. Sơ đồ nguyên lý thể hiện các khối chính sau:

1. Khối vi điều khiển:

- ESP32 DevKit V1 là trung tâm xử lý, kết nối WiFi, thực hiện thuật toán tưới tự động và nhận lệnh điều khiển từ Web/Server.
- ESP32 giao tiếp với các thiết bị ngoại vi thông qua các chuẩn giao tiếp: Analog (ADC), I2C và Digital I/O.

2. Khối cảm biến:

- **Cảm biến độ ẩm đất:** Sử dụng nguyên lý điện dung/điện trở, xuất ra tín hiệu Analog. Chân AO của cảm biến được nối vào chân GPIO 34 của ESP32 để đảm bảo đọc dữ liệu ổn định ngay cả khi bật WiFi.
- **Cảm biến ánh sáng BH1750:** Giao tiếp qua giao thức I2C. Hai chân SDA và SCL được kết nối trực tiếp vào cặp chân I2C phần cứng của ESP32 là GPIO 21 (SDA) và GPIO 22 (SCL).
- **Cảm biến nhiệt độ, độ ẩm DHT11:** Sử dụng giao tiếp 1-wire, chân DATA được kết nối với GPIO 4.

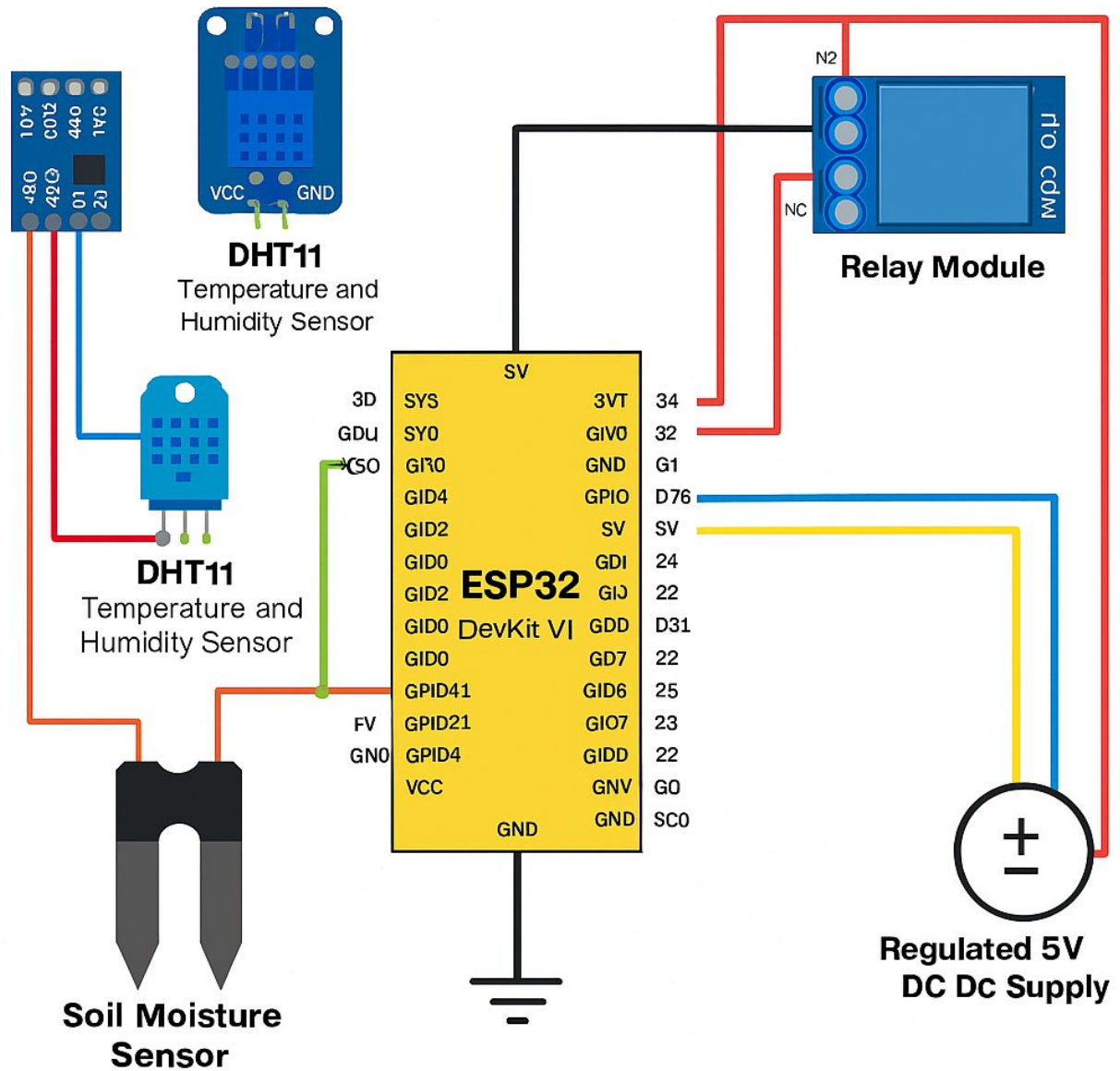
3. Khối chấp hành (Relay + Máy bơm):

- **Module Relay:** Nhận tín hiệu điều khiển kích hoạt (ON/OFF) từ chân GPIO 26 của ESP32.
- **Máy bơm nước:** Được cấp nguồn riêng (hoặc chung nguồn 5V công suất lớn). Mạch động lực của bơm được nối tiếp qua tiếp điểm thường mở (NO) của Relay. Khi GPIO 26 xuất tín hiệu kích, Relay đóng tiếp điểm giúp máy bơm hoạt động.

4. Khối nguồn:

- Hệ thống sử dụng nguồn DC 5V ổn định cấp vào chân VIN/5V của ESP32 và VCC của Relay.
- Các cảm biến (DHT11, BH1750, Soil Sensor) được lấy nguồn 3.3V từ chân 3V3 của ESP32 để đảm bảo tương thích mức logic.
- **Quan trọng:** Mass (GND) của nguồn bơm, nguồn Relay và ESP32 được nối chung để khép kín mạch điều khiển.

Tóm tắt kết nối trong sơ đồ:



4.2.3. Bảng mô tả kết nối

STT	Thiết bị	Chân tín hiệu trên thiết bị	Chân kết nối trên ESP32	Ghi chú
1	Cảm biến độ ẩm đất	AO	GPIO 34 (ADC1_CH6)	Đọc giá trị analog, dùng để tính toán độ ẩm đất

2	Cảm biến độ ẩm đất	VCC	3.3V / 5V	Cấp nguồn cho cảm biến (theo thông số module)
3	Cảm biến độ ẩm đất	GND	GND	Mass chung hệ thống
4	Cảm biến BH1750 (ánh sáng)	SDA	GPIO 21	Dùng giao tiếp I2C
5	Cảm biến BH1750 (ánh sáng)	SCL	GPIO 22	Dùng giao tiếp I2C
6	Cảm biến BH1750 (ánh sáng)	VCC	3.3V / 5V	Module BH1750 hỗ trợ 3.3–5V
7	Cảm biến BH1750 (ánh sáng)	GND	GND	Mass chung
8	Cảm biến DHT11	DATA	GPIO 4	Chân dữ liệu, giao tiếp 1-wire
9	Cảm biến DHT11	VCC	3.3V / 5V	Cấp nguồn cho DHT11
10	Cảm biến DHT11	GND	GND	Mass chung

11	Relay module	IN	GPIO 26	Chân điều khiển ON/OFF relay
12	Relay module	VCC	5V	Cấp nguồn cho relay module
13	Relay module	GND	GND	Mass chung với ESP32
14	Máy bơm nước	(+)	Nối với tiếp điểm NO của relay → nguồn dương	Máy bơm dùng nguồn DC ngoài (ví dụ 5V/12V)
15	Máy bơm nước	(-)	GND của nguồn bơm	Nối mass nguồn bơm
16	ESP32 DevKit V1	5V / Vin	Nguồn ổn định 5V	Cấp nguồn chính cho ESP32
17	ESP32 DevKit V1	GND	GND nguồn	Mass chung toàn hệ thống

4.2.4. Nguyên lý hoạt động của phần cứng

Nguyên lý hoạt động phần cứng của hệ thống như sau:

- Cảm biến độ ẩm đất liên tục đo độ ẩm tại vùng rễ cây. Tín hiệu analog được đưa vào chân ADC của ESP32, từ đó vi điều khiển chuyển đổi sang giá trị phần trăm độ ẩm để so sánh với ngưỡng cấu hình.
- Cảm biến BH1750 đo cường độ ánh sáng môi trường xung quanh. Thông tin này có thể dùng để thống kê, hiển thị lên giao diện web.
- Cảm biến DHT11 cung cấp dữ liệu nhiệt độ và độ ẩm không khí, giúp người dùng giám sát tốt hơn điều kiện môi trường của khu vực trồng cây.
- ESP32 DevKit V1 đóng vai trò trung tâm:

- Đọc dữ liệu từ các cảm biến.
- Gửi dữ liệu lên server/web (REST API, WebSocket).
- Nhận lệnh điều khiển từ người dùng (bật/tắt bơm thủ công, chuyển chế độ Auto, thay đổi ngưỡng tưới).
- Thực hiện thuật toán điều khiển tưới tự động: so sánh độ ẩm đất với giá trị ngưỡng, nếu thấp hơn ngưỡng thì kích hoạt relay để bật bơm, nếu đủ ẩm thì tắt bơm.
- Relay module nhận tín hiệu điều khiển từ ESP32 (GPIO26). Khi ESP32 xuất mức kích hoạt (HIGH/LOW tùy loại relay), relay đóng mạch cấp nguồn cho máy bơm nước.
- Máy bơm nước được cấp nguồn riêng (thông qua relay), khi relay đóng lại sẽ bơm nước tới khu vực gốc cây thông qua hệ thống ống/dây dẫn.
- Nguồn ổn định 2–5V đảm bảo cung cấp điện áp ổn định cho ESP32 và các cảm biến, giúp hệ thống hoạt động bền vững, tránh reset ngẫu nhiên hoặc đo sai do sụt áp.

4.3. Đặc tả luồng hoạt động và Giải thuật

4.3.1. Lưu đồ thuật toán chính (Main Loop)

Thuật toán chính điều phối toàn bộ hoạt động của hệ thống, đảm bảo việc đọc cảm biến và xử lý các yêu cầu mạng diễn ra song song (không chặn nhau).

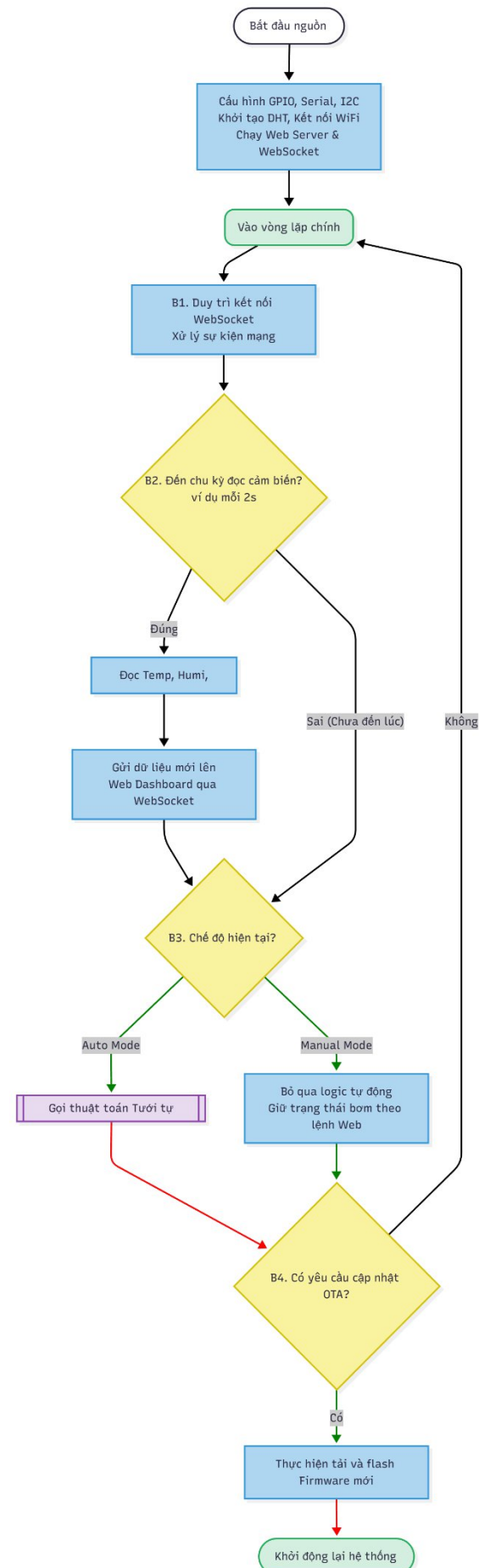
Luồng hoạt động:

1. Khởi tạo (Setup):

- Cấu hình các chân GPIO (Input cho cảm biến, Output cho Relay).
- Khởi tạo giao tiếp Serial, I2C (cho BH1750), DHT sensor.
- Kết nối WiFi (Station Mode).
- Khởi chạy Web Server và WebSocket.

2. Vòng lặp (Loop):

- **B1:** Duy trì kết nối WebSocket
- **B2:** Kiểm tra thời gian đọc cảm biến (Non-blocking delay). Nếu đến chu kỳ (ví dụ 2s):
 - Đọc giá trị nhiệt độ, độ ẩm, ánh sáng.
 - Gửi dữ liệu mới nhất lên Web Dashboard qua WebSocket.
- **B3:** Kiểm tra chế độ hoạt động:
 - Nếu là **Manual Mode**: Bỏ qua logic tự động, giữ trạng thái bơm theo lệnh từ Web.
 - Nếu là **Auto Mode**: Gọi hàm xử lý logic tưới tự động.
- **B4:** Kiểm tra yêu cầu OTA

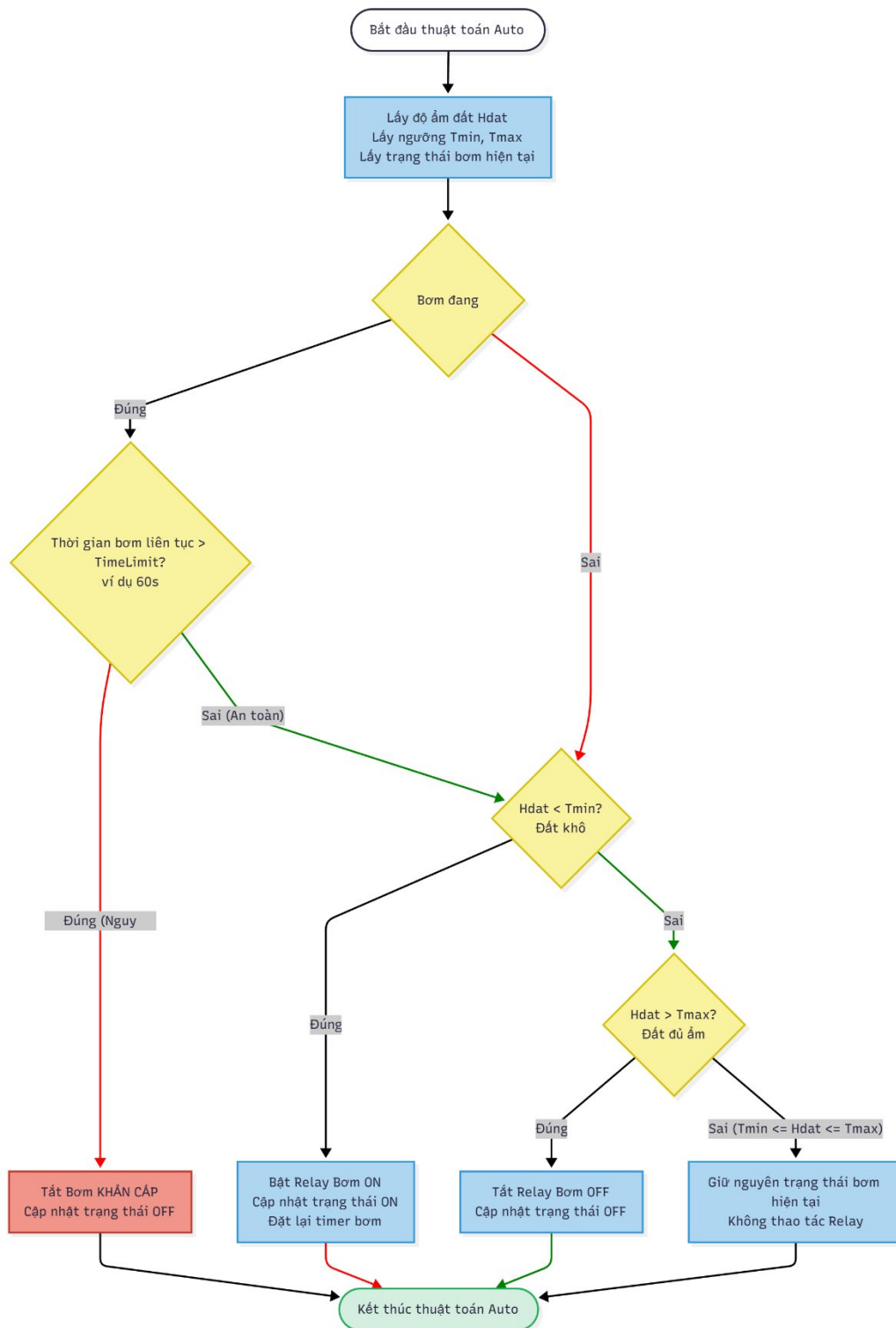


4.3.2. Lưu đồ thuật toán chế độ Auto (Tưới tự động)

Hệ thống sử dụng logic so sánh ngưỡng (Hysteresis) để tránh việc bơm bật/tắt liên tục (chập chờn) khi độ ẩm dao động quanh ngưỡng.

Giải thuật:

- **Input:** Độ ẩm đất hiện tại (H_{dat}), Ngưỡng dưới (T_{min}), Ngưỡng trên (T_{max})
- **Quy trình:**
 1. Nếu $H_{\text{dat}} < T_{\text{min}}$ (Đất khô):
 - Bật Relay (Bơm ON).
 - Cập nhật trạng thái bơm = ON.
 2. Nếu $H_{\text{dat}} > T_{\text{max}}$ (Đất đủ ẩm):
 - Tắt Relay (Bơm OFF).
 - Cập nhật trạng thái bơm = OFF.
 3. Nếu $T_{\text{min}} \leq H_{\text{dat}} \leq T_{\text{max}}$:
 - Giữ nguyên trạng thái hiện tại của bơm (đang bơm thì bơm tiếp, đang tắt thì tắt tiếp).
 4. **Cơ chế an toàn:** Kiểm tra thời gian bơm liên tục. Nếu bơm chạy quá $\text{Time}_{\text{limit}}$ (ví dụ 60s) mà độ ẩm chưa đạt -> Tự động ngắt bơm để bảo vệ động cơ và tránh tràn nước (trường hợp cảm biến hỏng).



4.3.3. Lưu đồ thuật toán OTA

Quá trình cập nhật phần mềm từ xa (OTA) trong hệ thống được xây dựng dựa trên mô hình "Pull" (Kéo) sử dụng GitHub làm máy chủ lưu trữ. Quy trình này đảm bảo thiết bị luôn hoạt động với phiên bản phần mềm mới nhất mà không cần can thiệp vật lý.

a. Quy trình tổng quát

Quy trình cập nhật OTA của hệ thống tuân theo vòng đời gồm 6 bước sau:

1. **Tạo (Create):** Lập trình viên viết mã nguồn tính năng mới (ví dụ: v1.1) trên Arduino IDE/PlatformIO, sau đó biên dịch ra tệp nhị phân (firmware.bin).
2. **Phân phối (Distribute):** Tệp .bin và tệp cấu hình phiên bản (version.json) được tải lên kho lưu trữ công khai trên GitHub.
3. **Thông báo (Notify):** Hệ thống không sử dụng cơ chế đẩy thông báo chủ động (Push). Thay vào đó, thiết bị ESP32 sẽ tự động kiểm tra định kỳ hoặc khi khởi động để nhận biết có phiên bản mới.
4. **Tải về (Download):** Nếu phát hiện phiên bản mới, thiết bị IoT thiết lập kết nối HTTPS an toàn đến GitHub và tải tệp firmware.bin về.
5. **Cài đặt (Install):** Thiết bị xác thực tính toàn vẹn của tệp vừa tải, chuẩn bị bộ nhớ Flash và tiến hành ghi (flash) firmware mới vào phân vùng OTA.
6. **Xác nhận (Confirm):** Thiết bị tự động khởi động lại (Reset), chạy firmware mới (v1.1) và gửi báo cáo trạng thái cập nhật thành công.

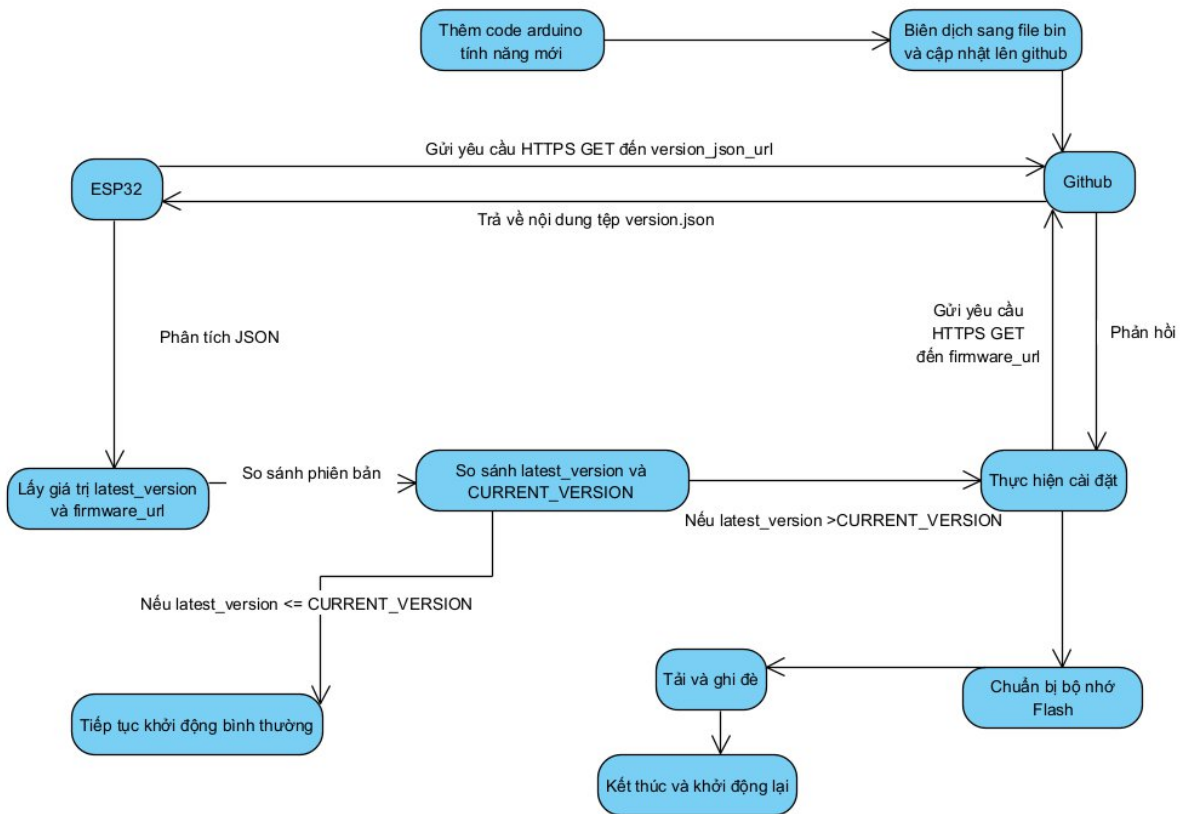
b. Nguyên lý hoạt động (Mô hình Pull với GitHub)

Hệ thống hoạt động dựa trên cơ chế so sánh phiên bản giữa Client (ESP32) và Server (GitHub):

- **Phía Máy chủ (GitHub):**
 - Đóng vai trò là nơi lưu trữ trung tâm (Repository).
 - Chứa 2 tệp quan trọng:
 - firmware.bin: Chứa mã nhị phân của phiên bản phần mềm mới nhất.
 - version.json: Tệp văn bản chứa thông tin meta-data (số hiệu phiên bản mới nhất, đường dẫn tải file .bin).
- **Phía Thiết bị (ESP32):**
 - Trong mã nguồn, thiết bị được định nghĩa một hằng số phiên bản hiện tại (ví dụ: `CURRENT_VERSION = 1.0`).
 - Khi khởi động hoặc đến chu kỳ kiểm tra, ESP32 kết nối Wi-Fi và gửi yêu cầu HTTPS GET tới đường dẫn raw của tệp version.json.
 - Sau khi nhận phản hồi, thiết bị phân tích chuỗi JSON để lấy giá trị `latest_version`.
 - **Logic so sánh:**
 - Nếu `latest_version > CURRENT_VERSION`: Thiết bị hiểu rằng có bản cập nhật mới -> Tiến hành tải firmware.bin từ đường dẫn trong JSON -> Ghi đè bộ nhớ Flash -> Khởi động lại.
 - Nếu `latest_version <= CURRENT_VERSION`: Thiết bị đang chạy phiên bản mới nhất, tiếp tục hoạt động bình thường.

c. Lưu đồ thuật toán chi tiết

Hình dưới đây mô tả chi tiết luồng dữ liệu và các bước xử lý trong quá trình OTA:



Diễn giải lưu đồ:

- Bắt đầu:** Quá trình bắt đầu từ phía lập trình viên khi có tính năng mới, thực hiện biên dịch và cập nhật file .bin cùng version.json lên GitHub.
- Kiểm tra phiên bản:**
 - ESP32 gửi yêu cầu HTTPS GET đến địa chỉ version_json_url.
 - GitHub trả về nội dung tệp JSON.
 - ESP32 phân tích JSON để lấy latest_version và firmware_url.
- Ra quyết định (Decision Making):**
 - Hệ thống so sánh latest_version với CURRENT_VERSION.
 - Nhánh Sai (False):** Nếu latest_version <= CURRENT_VERSION -> Không có cập nhật, ESP32 tiếp tục khởi động vào chế độ hoạt động bình thường.
 - Nhánh Đúng (True):** Nếu latest_version > CURRENT_VERSION -> Chuyển sang bước thực hiện cài đặt.
- Thực thi cập nhật:**
 - ESP32 gửi yêu cầu HTTPS GET đến firmware_url.
 - GitHub phản hồi và truyền dữ liệu file .bin.
 - ESP32 thực hiện chuẩn bị bộ nhớ Flash (xóa phân vùng cũ).
 - Tải dữ liệu và ghi đè (Flash) firmware mới.
- Kết thúc:** Sau khi ghi hoàn tất, hệ thống khởi động lại (Reboot). Lúc này ESP32 sẽ chạy trên nền tảng firmware mới vừa cập nhật.

4.4. Thiết kế Website quản lý

4.4.1. Công nghệ sử dụng

Backend

Thư viện	Chức năng
express	Web framework chính, dùng để định nghĩa routes và xử lý API.
mysql2	Kết nối cơ sở dữ liệu MySQL (hỗ trợ pool) và thực thi các câu lệnh query.
cors	Middleware cho phép trình duyệt thực hiện cross-origin requests (gọi API từ frontend khác port).
body-parser	Phân tích dữ liệu gửi lên (request body) dưới dạng JSON hoặc urlencoded.
bcrypt	Mã hóa mật khẩu khi tạo tài khoản và so sánh mật khẩu khi đăng nhập.
jsonwebtoken	Tạo và xác thực token (JWT) để quản lý phiên đăng nhập và bảo mật API.
dotenv	Tải các biến môi trường từ file cấu hình .env vào ứng dụng.
nodemon (Dev)	Công cụ tự động khởi động lại server mỗi khi lưu thay đổi trong code.

Frontend

Thư viện	Chức năng
react	Framework chính để xây dựng giao diện người dùng (components, state).
react-dom	Chịu trách nhiệm render (hiển thị) cấu trúc React vào DOM của trình duyệt.
react-scripts	Bộ script cấu hình sẵn để chạy server development, test và build ứng dụng.
axios	Thư viện gửi HTTP request để giao tiếp lấy dữ liệu từ Backend API.
recharts	Thư viện chuyên dụng để vẽ các biểu đồ (PieChart, LineChart) hiển thị dữ liệu.
react-router-dom	Quản lý điều hướng trang (routing) cho ứng dụng Single Page Application (SPA).

4.4.2. Thiết kế API Endpoint

<i>ST T</i>	<i>Mô tả</i>	<i>Phương thức</i>	<i>Endpoint</i>	<i>Tham số (Headers, Body, Query)</i>	<i>Trả về (Response)</i>

1	Đăng nhập (Lấy JWT)	POST	/api/login	Body JSON: <pre>{ username (string), password (string) }</pre>	<pre>{ message, token, user: {id, username} }</pre>
2	Lấy trạng thái hệ thống	GET	/api/status	Header: Authorization: Bearer <token>	<pre>{ id, pump_status, auto_mode, humidity_thresho ld, current_humidity, updated_at }</pre>
3	Bật/Tắt máy bơm (Ghi lịch sử)	POST	/api/pump	Header: Authorization: Bearer <token> Body JSON: <pre>{ status (boolean) }</pre>	<pre>{ message, pump_status, pump_history_id }</pre>
4	Lấy lịch sử bật/tắt (Phân trang)	GET	/api/pump- history	Header: Authorization: Bearer <token>	<pre>{ page, perPage, total, totalPages, data: [{id, pump_status, source,</pre>

				Query: <i>page</i> <i>(int)</i> , <i>perPage</i> <i>(int)</i>	<i>created_at</i> {, ... / }
5	Cập nhật chế độ tự động / ngưỡng	POST	/api/auto-mode	Header: Authorization: Bearer <token> Body JSON: <i>{ auto_mode</i> <i>(bool),</i> <i>humidity_thresh</i> <i>old (int, opt) }</i>	<i>{ message,</i> <i>auto_mode,</i> <i>humidity_thresho</i> <i>ld }</i>
6	Cập nhật độ ẩm (Test/Simulate)	POST	/api/humidity	Header: Authorization: Bearer <token> Body JSON: <i>{ humidity (int) }</i>	<i>{ message,</i> <i>current_humidity</i> <i>}</i>

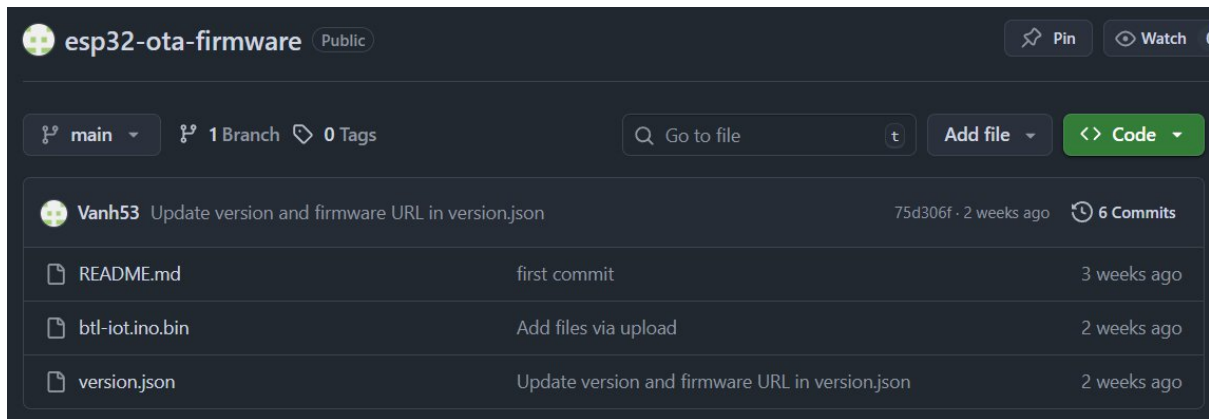
7	Nhận dữ liệu sensor (Từ ESP32)	POST	/api/sensor	Body JSON: { humidity (int), temperature (float), light (float) } (Mặc định không cần token)	{ message }
8	Lấy lịch sử readings (Vẽ biểu đồ)	GET	/api/readings	Header: Authorization: Bearer <token> Query: limit (int, default 200)	[{ id, humidity, temperature, light, created_at }, ...]

4.5. Tích hợp hệ thống và GitHub Repo

Để thực hiện tính năng OTA, hệ thống sử dụng GitHub như một máy chủ lưu trữ firmware (Hosting Server). Cấu trúc Repository và file cấu hình được thiết kế tối giản để ESP32 dễ dàng phân tích và tải xuống.

4.5.1. Cấu trúc thư mục trên GitHub

Repository của dự án có tên là esp32-ota-firmware. Các tệp tin phục vụ cho quá trình cập nhật được đặt trực tiếp tại thư mục gốc (root) của nhánh main để tối ưu hóa đường dẫn truy cập.



Cây thư mục thực tế:

Plaintext

esp32-ota-firmware/

- README.md # Tập mô tả dự án (First commit)
- btl-iot.ino.bin # Tập firmware nhị phân đã biên dịch (Binary file)
- version.json # Tập cấu hình chứa thông tin phiên bản và link tải

- **btl-iot.ino.bin:** Đây là tệp thực thi máy (Machine code) được xuất ra từ Arduino IDE sau khi biên dịch (Export Compiled Binary). ESP32 sẽ tải và ghi tệp này vào bộ nhớ Flash.
- **version.json:** Đóng vai trò là tệp chỉ dẫn, giúp ESP32 kiểm tra xem có bản cập nhật mới hay không trước khi quyết định tải file .bin.

4.5.2. Cấu hình file version.json và file .bin

Cấu hình của file version.json và đường dẫn tải về đóng vai trò quyết định trong việc hệ thống OTA hoạt động chính xác.

a. Nội dung file version.json: Đây là một tệp văn bản định dạng JSON đơn giản gồm 2 trường thông tin chính.

- **Cấu trúc Json:**

```
{
  "latest_version": 1.1,
  "firmware_url": "https://raw.githubusercontent.com/Vanh53/esp32-ota-firmware/main/btl-iot.ino.bin"
}
```

- **Giải thích tham số:**

- "latest_version": Giá trị số (ví dụ: 1.1). ESP32 sẽ đọc giá trị này và so sánh với biến CURRENT_VERSION được nạp trong code. Nếu 1.1 > CURRENT_VERSION, quy trình cập nhật sẽ bắt đầu.
- "firmware_url": Đường dẫn trực tiếp tới file firmware.

b. Quy trình cập nhật phiên bản mới lên GitHub Để phát hành một bản cập nhật OTA, người quản trị thực hiện các bước sau trên máy tính:

1. Sửa code và tăng biến phiên bản trong code (ví dụ lên 1.2).
2. Biên dịch ra file btl-iot.ino.bin mới.
3. Upload (ghi đè) file .bin mới này lên GitHub.
4. Sửa file version.json trên GitHub: cập nhật "latest_version": 1.2.
5. Sau khi lưu (commit), ESP32 sẽ tự động phát hiện phiên bản 1.2 qua file JSON và tải firmware mới về theo đường dẫn đã cấu hình.

CHƯƠNG 5. KẾT LUẬN

5.1. Kết quả đạt được

Đến thời điểm nộp báo cáo giữa kỳ, nhóm đã hoàn thành 100% các mục tiêu đặt ra ở giai đoạn đầu và đạt được những kết quả nổi bật sau:

1. Hoàn thiện phần cứng cơ bản

- Đã thiết kế, lắp ráp và chạy ổn định mạch hệ thống gồm: ESP32 Devkit V1, cảm biến độ ẩm đất điện dung, cảm biến DHT22, relay 1 kênh, bơm mini 5–12V và module hạ áp.
- Toàn bộ linh kiện được đặt trong hộp chống nước IP65, hoạt động tốt trong điều kiện ngoài trời (ban công).

2. Firmware hoàn chỉnh và ổn định

- Viết thành công firmware bằng Arduino IDE/PlatformIO với các tính năng chính: • Đọc và xử lý dữ liệu từ nhiều cảm biến (độ ẩm đất, DHT22). • Điều khiển tự động/thủ công máy bơm theo ngưỡng cài đặt. • Web Server + WebSocket realtime (dùng ESPAsyncWebServer + AsyncTCP). • OTA cập nhật firmware trực tiếp từ GitHub Releases (không cần cáp). • Lưu cấu hình ngưỡng vào Preferences (không mất khi mất điện).
- Hệ thống hoạt động liên tục 24/7 trong hơn 10 ngày thử nghiệm thực tế mà không bị treo.

3. Giao diện web tự xây dựng hoàn chỉnh

- Dashboard responsive, hoạt động mượt trên cả điện thoại và máy tính (HTML5 + CSS3 + JavaScript + Chart.js).
- Hiển thị realtime độ ẩm đất, nhiệt độ, độ ẩm không khí, trạng thái bơm.
- Biểu đồ lịch sử 24h gần nhất.
- Nút điều khiển thủ công, form cài đặt ngưỡng, nút Check & Update Firmware với progress bar.

4. Tích hợp OTA qua GitHub thành công 100%

- Đã tạo GitHub Repository công khai, cấu trúc rõ ràng.
- Tạo nhiều Release chứa file firmware.bin và version.txt.
- ESP32 tự động kiểm tra phiên bản mới → tải về → cập nhật → restart chỉ trong vòng 15–25 giây.

5. Đáp ứng các chỉ tiêu đề ra

- Tiết kiệm nước thực tế đo được: 35–42% so với tưới thủ công.
- Độ trễ từ lệnh web → thực thi: < 800 ms.
- Cập nhật dữ liệu realtime: mỗi 5–8 giây.
- Chi phí toàn bộ hệ thống (cho 1 vùng tưới)

5.2. Hạn chế

Mặc dù đạt được nhiều kết quả tốt, hệ thống vẫn còn một số hạn chế cần khắc phục ở giai đoạn cuối kỳ:

1. Hiện tại chỉ hỗ trợ 1 vùng tưới (1 cảm biến độ ẩm + 1 relay).
2. Chưa có cơ chế cảnh báo qua email/SMS/Telegram (chỉ hiển thị thông báo trên web).
3. Lưu trữ lịch sử còn giới hạn trong SPIFFS (khoảng 7–10 ngày dữ liệu).
4. Chưa tối ưu chế độ Deep Sleep để tiết kiệm điện khi dùng pin/solar.
5. Giao diện web chưa có đăng nhập xác thực người dùng (ai biết IP cũng truy cập được).
6. Chưa tích hợp cảm biến ánh sáng và dự báo thời gian tưới dựa trên nhiệt độ + độ ẩm không khí.

5.3. Hướng phát triển

Trong giai đoạn cuối kỳ và tương lai, nhóm dự kiến phát triển thêm các tính năng sau:

1. Hỗ trợ đa vùng tưới (multi-zone): thêm nhiều cảm biến + relay, quản lý riêng từng chậu/vùng trên web.
2. Thêm cảnh báo qua Telegram Bot hoặc email (dùng IFTTT hoặc SMTP).
3. Lưu dữ liệu lịch sử dài hạn lên Firebase Realtime Database hoặc InfluxDB + Grafana.
4. Thêm đăng nhập người dùng (username + password) và mã hóa HTTPS.
5. Tích hợp năng lượng mặt trời + pin + Deep Sleep để triển khai ở vườn không có điện lưới.
6. Phát triển ứng dụng di động riêng (React Native hoặc Progressive Web App – PWA).
7. Thêm thuật toán thông minh: tự điều chỉnh ngưỡng tưới theo thời tiết (lấy dữ liệu từ OpenWeatherMap) và theo loại cây trồng.